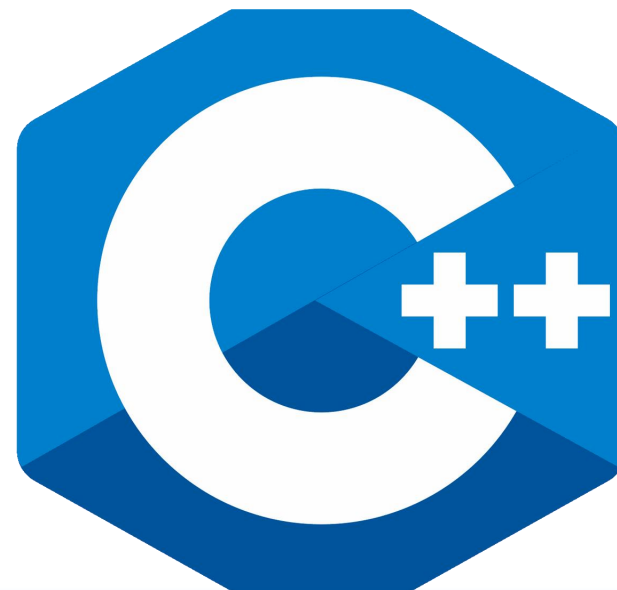


第一课：C++ 高级程序设计概述

李正丹

2025



目录

1. 教材 & 参考资料

- Thinking in C++
- C++ Primer Plus
- 高级程序 C++ 语言设计 (二)
- C++ 面向对象程序设计

2. C++ 简介

- C++ 版本与特点
- 现实中的 C++ 应用
- 本课程的学习方法

3. 一个 C++ 程序示例

- C++ 开发环境
- 示例代码
- 编译与运行

4. 面向对象编程 (OOP) 简介

- OOP 的特点
- 什么是 class
- 设计一个 class
- 实例化并进行操作

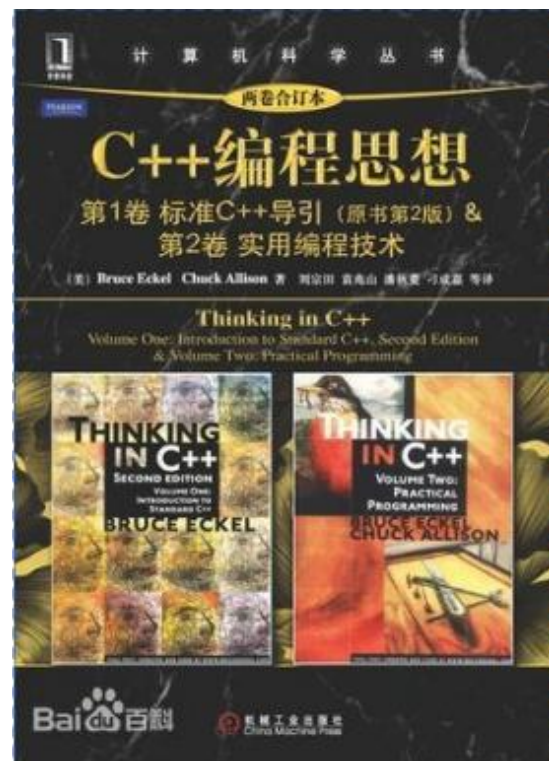
为什么要学 C++

- 了解计算机的底层工作原理
- 从计算机的角度考虑问题解决问题
- 为学习其他知识奠定基础

教材 & 参考资料

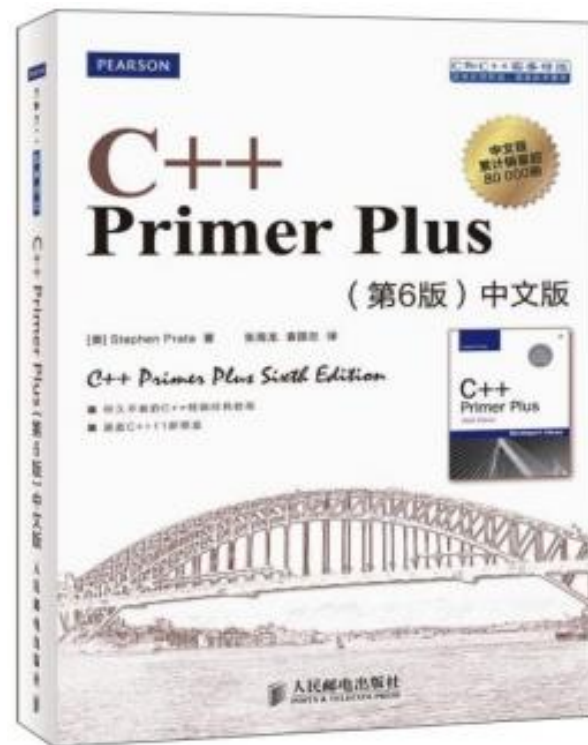
Thinking in C++

- 对象导言
- 对象的创建与应用
- C++ 中的 C
- 数据抽象
- 隐藏实现
- 初始化与清除
- ...
- 多态与虚函数
- 模板介绍



C++ Primer Plus

- 预备知识
- 开始学习 C++
- 处理数据
- 复合类型
- 循环和关系表达式
- 分支语句和逻辑运算符
- C++ 中的代码重用
- 友元、异常和其他



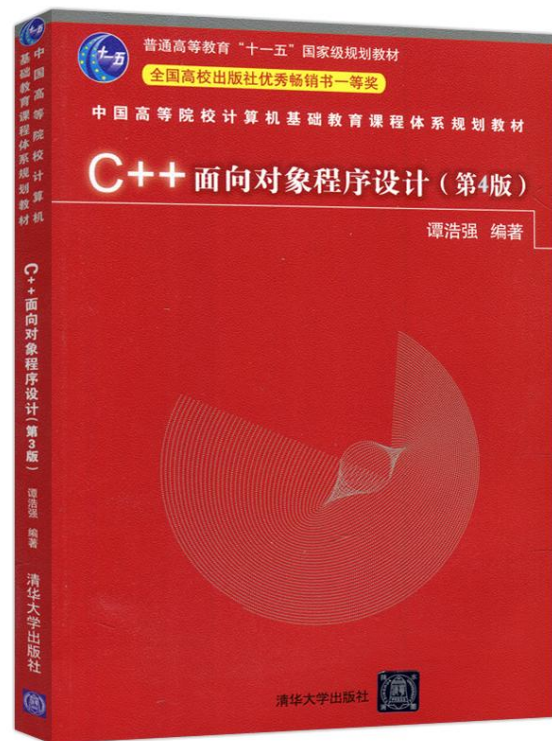
高级程序 C++ 语言设计(二)

- C 语言部分
- 类与对象
- 继承与派生
- 模板
- 输入输出流



面向对象程序设计

- C++的初步认识
- 类和对象的特性
- 怎么使用类和对象
- 对运算符进行重载
- 类的继承
- ...



C++ 简介

C++ 版本与特点

- C++ 的兴起
 - 1985-1994: 使用人数爆炸性增长
 - 1995-2004: 稳定增长, 2003年达到300万C++程序员
- C++ 的版本迭代
 - C++98/C++03
 - 第一个内部标准
 - 包括了 STL 库。
 - C++11
 - C++14
 - C++17
 - C++20
 - C++23/2b

- C++ 复杂的根源在于三大约束
 - 与C的完全兼容
 - 静态类型检查
 - 最高性能
- 缺点：
 - 复杂导致学习困难，开发效率低
 - 企业级软件不会使用 C++(注重开发效率)
 - 系统级软件:C/C++ 是首选(注重性能)

现实中的 C++ 应用

- Adobe: Photoshop, Illustrator, Acrobat
- Maya
- Autodesk
- Amazon.com
- Google
- IBM: OS/400
- Intel: 编译器和优化器
- Microsoft: Windows XP, NT, Office, IE, VC, C# 编译器
- MySQL, DOOM III, KDE

本课程的学习方法

- 理解 C++ 语言特性
 - 避免 C 语言的缺点(面向过程编程)。
 - 抽象更高效的编程方案(面向对象)
- 熟悉 C++ 的知识结构
 - C 语言的基本语法
 - 基本数据类型与导出数据类型
 - 基本控制语法
 - 函数与函数重载
 - 指针、引用、动态内存分配

本课程的学习方法（续）

- 熟悉 C++ 的知识结构
 - 面向对象的抽象特性
 - 类与对象
 - 构造与析构函数
 - 封装、继承、多态
 - 友元/静态/常量函数和变量
 - 模板函数/类
- 听课、看书、编程实践
 - 菜鸟 C++ 教程
 - C++ 在线编译测试工具

一个 C++ 程序示例

C++ 开发环境

- Editor + Compiler
 - Editor: vim, VSCode, Atom
 - Compiler: gcc, clang
- IDE: Eclipse, Visual Studio, XCode
- Online: C++ 在线编译工具, LeetCode

示例代码

course1.cpp

```
#include <string>
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream is("test.txt");
    string line;
    while (getline(is, line)) {
        cout << line << endl;
    }
}
```

test.txt

High

level

language

C++

program

design

使用 g++ 编译器进行编译

- 验证 g++ 是否安装: `g++ --version`
- 编译源码: `g++ course1.cpp`
- 运行程序: `./a.out`

面向对象编程 (OOP) 简介

OPP的特点

- class: 具有相同属性和操作的对象的抽象描述, 用于描述某一类对象的共同特征和行为。
- object: 是一个类(type)的实例, 任何事物都是对象:数据+ 操作的集合
- message: 调用对象的成员函数完成操作, 称为向这个对象发消息

什么是 class

- Definition: describes a set of objects that have identical characteristics(data elements) and behaviors (functionality)。
- Examples:电灯泡是这样的一类物品:具有额定工作电压(voltage), 具有打开(on)和关闭(off)发光操作, 可以调节亮(brighten)暗(dim)操作。

示例代码: Light 类定义

Light类具有的数据和操作

- Light类定义

```
class Light {  
private:  
    float voltage; // 额定工作电压  
public:  
    bool on();      // 打开(on)发光操作  
    bool off();     // 关闭(off)发光操作  
    bool brighten(); // 调亮(brighten)操作  
    bool dim();     // 调暗(dim)操作  
}
```


示例代码: Light 类定义

- 实例化Light类

```
int main() {  
    Light classroom_light;  
    classroom_light.voltage = 220;  
  
    classroom_light.on();  
    classroom_light.brighten();  
}
```

使用标准类库的对象

目录

- `iostream(cout/cin)`
- `string`
- `ifstream` 和 `ofstream`
- `vector`

cout/cin简介

- cout/cin 是 C++ 输出/输入流 (iostream) 对象, - cout/cin 能够操纵数据流
- 运算符 (<>) 可以被重载

简单的 cout 示例

```
// simple usage of cout
#include <iostream>
using namespace std;

int main() {
    cout << "A dog has" << 4 << " legs" << endl;
}
```

指定 cout 的输出格式

```
// decimal, octal, hexadecimal, ASCII
#include <iostream.h>
using namespace std;
int main() {
    // Specifying formats with manipulators:
    cout << "a number in decimal: " << dec << 15 << endl;
    cout << "in octal: " << oct << 15 << endl; //17
    cout << "in hex: " << hex << 15 << endl; //F
    cout << "a floating-point number: " << 3.14159 << endl;
    cout << "char (A): " << char(65) << endl;
}
```

简单的 cin 示例

```
#include <iostream.h>
using namespace std;
int main() {
    int number;
    cout << "Enter a decimal number: ";
    cin >> number;
    cout << "value in octal = 0" << oct << number << endl;
    cout << "value in hex = 0x" << hex << number << endl;
}
```

string: 字符串标准库

string 标准库隐藏了操作字符数组的实现细节，可以方便地进行字符数组操作，如赋值、连接、追加等。

简单的 string 示例

```
#include <string>
#include <iostream>
using namespace std;

int main() {
    string s1, s2; // Empty strings
    string s3 = "Hello, World."; // Initialized
    string s4("I am"); // Also initialized
    s2 = "Today"; // Assigning to a string
    s1 = s3 + " " + s4; // Combining strings
    s1 += " 8 "; // Appending to a string
    cout << s1 + s2 + "!" << endl;
}
```


ifstream 和 ofstream 使用简单示例

ifstream 和 ofstream 是文件输入输出流对象，用来读写文件。

```
#include <string>
#include <fstream>
using namespace std;

int main() {
    ifstream in("input.txt"); // Open for reading
    ofstream out("output.txt"); // Open for writing
    string s;
    while (getline(in, s)) // Discards newline char
        out << s << "\n"; // must add it back
}
```

string 字符串动态增长示例

string 字符串对象可以根据字符长度而动态调整长度，从而避免字符串溢出，示例如下：

```
#include <string>
#include <iostream>
#include <fstream>
int main() {
    ifstream in("FillString.cpp");
    string s, line;
    while(getline(in, line))
        s += line + "\n";
    cout << s;
}
```

vector 简介

- vector 是一种可以动态增长的数组
- 类模板
 - C++ 标准库中的一个类模板
 - 常见类型包括 `vector <string>`、`vector <int>`、`vector <double>`
- 常用操作包括 `push_back` 和索引操作。

vector 示例 1: 追加数据和读取数据

```
#include <string>
#include <iostream>
#include <fstream>
#include <vector>
using namespace std;

int main() {
    vector<string> v;
    ifstream in("Fillvector.cpp");
    string line;
    while(getline(in, line))
        v.push_back(line); // Add the line to the end
    // Add line numbers:
    for(int i = 0; i < v.size(); i++)
        cout << i << ": " << v[i] << endl;
}
```

vector 示例 2: 计算 vector 内数据并写回

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v;
    for(int i = 0; i < 10; i++)
        v.push_back(i);
    for(int i = 0; i < v.size(); i++)
        cout << v[i] << ", ";
    cout << endl;
    for(int i = 0; i < v.size(); i++)
        v[i] = v[i] * 10;
    for(int i = 0; i < v.size(); i++)
        cout << v[i] << ", ";
    cout << endl;
}
```

vector 示例 3：构建矩阵并操作

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<vector<double>> m(4, vector<double>(4));
    int cnt = 0;
    for(int i = 0; i < m.size(); i++)
        for(int j = 0; j < m[i].size(); j++)
            m[i][j] = cnt++;
    for(int i = 0; i < m.size(); i++) {
        for(int j = 0; j < m[i].size(); j++)
            cout << m[i][j] << " ";
        cout << endl;
    }
}
```